Assignment 4 - Variational Autoencoder (VAE)

April 01, 2026

Group 15

Sezal Rana (UD24016) Anoushka Waghmare (B24332) Akshat Jha (B23336)
Mohit Kulhari (S25016) Shubhankit Singh (B23387) Aashvi Garg (B24102)
Bhavya Sunkari (B23367) Harsh Vardhan Sharma (B23373) Madhav Aggrawal (B24036)

1 The Probabilistic Bottleneck

1.1 The Reparameterization Trick

1.1.1 Concept

In a standard Autoencoder the bottleneck is a **fixed vector** $z = f(x)$. A VAE replaces this with a **distribution**:

$$q_\phi(z \mid x) = \mathcal{N}\big(\mu,\, \sigma^2 I\big)$$

The encoder no longer outputs a single point; instead it outputs the *parameters* of a Gaussian—the mean μ and the log-variance $\log \sigma^2$—via two parallel linear heads.

1.1.2 Why Direct Sampling Fails

Sampling $z \sim \mathcal{N}(\mu, \sigma^2)$ is a stochastic operation. Gradients **cannot** flow through a random node, so standard backpropagation breaks at the bottleneck.

1.1.3 The Reparameterization Trick

Factor the randomness out as an *external* noise variable $\varepsilon \sim \mathcal{N}(0, I)$ that requires no gradient:

$$\boxed{z \;=\; \mu \;+\; \sigma \cdot \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0,\, I)}$$

$$\sigma = \exp\big(\tfrac{1}{2} \log \sigma^2\big)$$

Now z is a *deterministic* function of $(\mu, \sigma, \varepsilon)$, so the partial derivatives exist:

$$\frac{\partial z}{\partial \mu} = 1, \qquad \frac{\partial z}{\partial \sigma} = \varepsilon$$

Gradients flow cleanly back through both heads during training. At inference, ε is dropped and only μ is returned (deterministic reconstruction).

1.1.4 Encoder Architecture

Component	Detail
Encoder backbone	Linear $\rightarrow$ BN $\rightarrow$ ReLU $\rightarrow$ Linear $\rightarrow$ BN $\rightarrow$ ReLU
`fc_mu`	Linear($h/2$, d_z) — outputs μ
`fc_logvar`	Linear($h/2$, d_z) — outputs $\log \sigma^2$
Sampling layer	$z = \mu + \exp(0.5 \cdot \log \sigma^2) \cdot \varepsilon$

1.1.5 Implementation

Sampling Layer (`Reparameterize`)

```python
class Reparameterize(nn.Module):

    def forward(self, mu: torch.Tensor, log_var: torch.Tensor) -> torch.Tensor:
        if self.training:
            std = torch.exp(0.5 * log_var)
            eps = torch.randn_like(std)
            return mu + std * eps
        else:
            return mu
```

Code Snippet 1: Reparameterization / Sampling layer

Encoder with Dual Bottleneck Heads

```python
class Encoder(nn.Module):

    def __init__(self, input_dim: int, hidden_dim: int, latent_dim: int):
        super().__init__()

        self.shared = nn.Sequential(
            nn.Linear(input_dim, hidden_dim),
            nn.BatchNorm1d(hidden_dim),
            nn.ReLU(),
            nn.Linear(hidden_dim, hidden_dim // 2),
            nn.BatchNorm1d(hidden_dim // 2),
            nn.ReLU(),
        )

        # Two PARALLEL bottleneck heads
        self.fc_mu     = nn.Linear(hidden_dim // 2, latent_dim)
        self.fc_logvar = nn.Linear(hidden_dim // 2, latent_dim)

        self.reparameterize = Reparameterize()

    def forward(self, x: torch.Tensor):
        x_flat  = x.view(x.size(0), -1)
        h       = self.shared(x_flat)
        mu      = self.fc_mu(h)
        log_var = self.fc_logvar(h)
        z       = self.reparameterize(mu, log_var)
        return z, mu, log_var
```

Code Snippet 2: Encoder — shared trunk + two parallel linear heads

1.2 The Loss Function (KLD + Reconstruction)

Formulation of the VAE Loss

The Variational Autoencoder (VAE) optimizes a composite objective consisting of a reconstruction term and a regularization term based on Kullback-Leibler Divergence (KLD). The total loss is given by:

$$\mathcal{L}_{total} = \mathcal{L}_{rec} + \beta \cdot \mathcal{L}_{KLD}$$

For MNIST data, Binary Cross Entropy (BCE) was used as the reconstruction loss:

$$\mathcal{L}_{rec} = -\sum \left[x \log(\hat{x}) + (1-x)\log(1-\hat{x})\right]$$

The KLD term enforces the latent distribution to follow a standard normal distribution:

$$\mathcal{L}_{KLD} = -\frac{1}{2}\sum \left(1 + \log\sigma^2 - \mu^2 - \sigma^2\right)$$

Here, μ and $\log\sigma^2$ are the encoder outputs. The parameter β controls the trade-off between reconstruction fidelity and latent space regularization.

Loss Normalization

To improve interpretability, all losses were normalized per image by dividing the total loss accumulated over each epoch by the number of samples. This avoids large magnitudes arising from summation over all pixels and batches, and allows meaningful comparison across different values of β.

Training Behaviour

The model was trained for 20 epochs with three different values of $\beta \in \{0.1, 1, 5\}$. The total loss, reconstruction loss, and KLD loss were tracked separately.

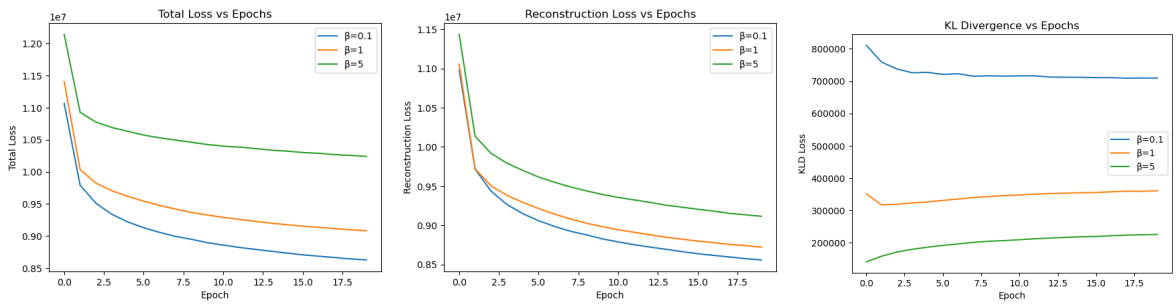


Figure 1: Total loss, reconstruction loss, and KL divergence over epochs for different β values

From the loss curves, the following observations can be made:

- The **total loss decreases steadily** for all values of β, indicating stable training.
- The **reconstruction loss is lowest for smaller** β and increases as β increases.

- The **KLD term is highest for smaller** β and decreases significantly for larger β, showing stronger regularization.

Quantitative Comparison

A comparison of final epoch losses is summarized in Table 1.

Table 1: Effect of KLD weight (β) on normalized losses (per image)

Beta (β)	**Total Loss**	**Reconstruction Loss**	**KLD Loss**
0.1	143.40	142.27	11.31
1	149.24	143.17	6.07
5	170.56	151.94	3.72

Reconstruction Analysis

The qualitative effect of varying β on reconstructed images is shown in Figure 2.

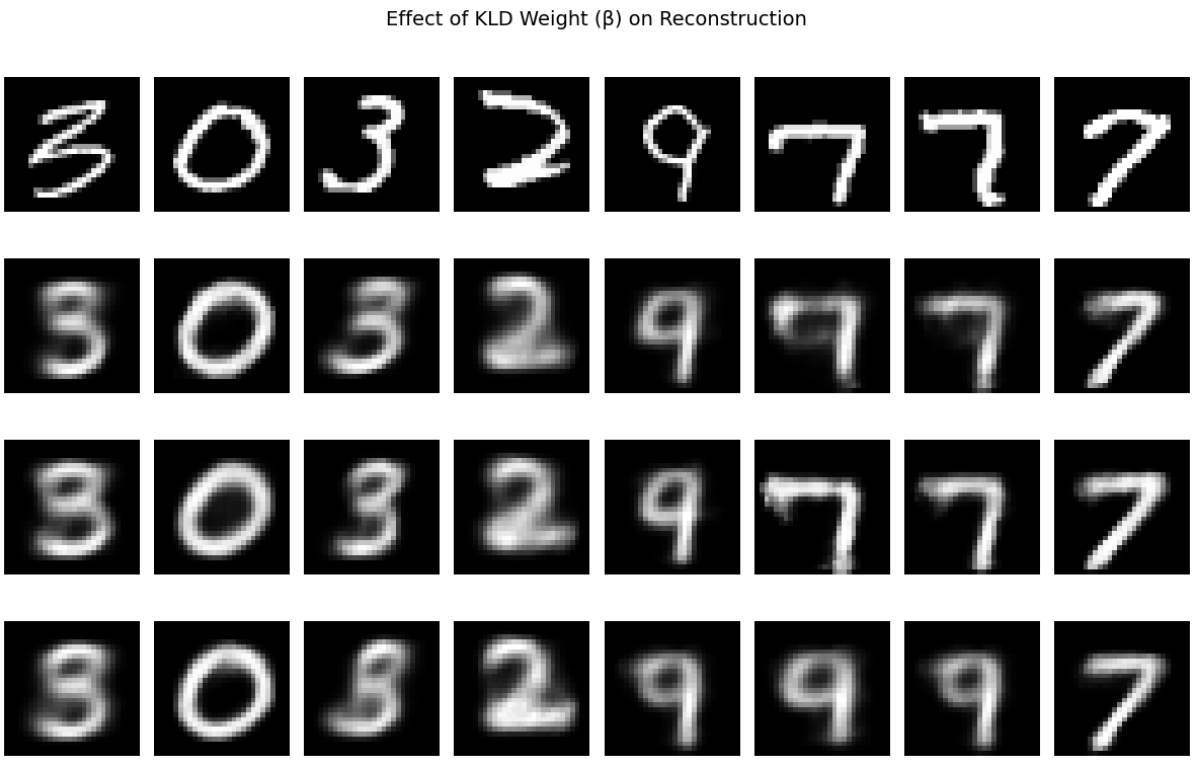


Figure 2: Effect of KLD weight (β) on reconstruction quality

From the reconstruction results:

- For $\beta = 0.1$, the reconstructions are the **sharpest**, preserving fine details.
- For $\beta = 1$, a **balanced trade-off** is observed between clarity and smoothness.

- For $\beta = 5$, the reconstructions become **noticeably blurred**, with smoother but less detailed structures.

Discussion

The results highlight the fundamental trade-off in VAEs:

- Lower values of β prioritize reconstruction accuracy, leading to sharper images but a poorly structured latent space.
- Higher values of β enforce stronger latent space regularization, resulting in smoother representations but degraded reconstruction quality.

Conclusion

The experiment confirms that increasing the weight of the KLD term reduces the sharpness of reconstructed images. This occurs because stronger regularization constrains the latent space, limiting the model's ability to preserve fine-grained details. Therefore, selecting an appropriate value of β is crucial to balance reconstruction fidelity and generative capability.

2 Latent Space Geometry

2.1 The Latent Walk (Interpolation)

Methodology

To explore the smoothness of the learned latent space, a linear interpolation was performed between the latent representations of two images from different classes in Tiny-ImageNet-10. Given two encoded latent vectors μ_1 and μ_2 (using the mean of the encoder distribution for stability), the interpolated latent vector is computed as:

$$z_{\text{new}} = \alpha \cdot \mu_1 + (1 - \alpha) \cdot \mu_2, \quad \alpha \in \{1.0, 0.9, 0.8, \ldots, 0.0\}$$

For each of the 10 values of α, the interpolated vector z_{new} was passed through the trained CVAE decoder to generate an intermediate image. This produces a transition strip morphing from Class A to Class B. Note that μ is used instead of a sampled z to ensure a deterministic and stable interpolation path through the latent space.

Selected Images

Two images were selected from different classes in the Tiny-ImageNet-10 test set, as shown in Figure **??**.

Figure 3: Two images selected from different classes for the latent walk experiment.

Transition Strip

The resulting transition strip showing the image morphing from Class A ($\alpha = 1.0$) to Class B ($\alpha = 0.0$) across 10 interpolation steps is shown in Figure 4.

Figure 4: Q2.1 — Latent walk transition strip: morphing from Class A (food, $\alpha = 1.0$) to Class B (mountain/snow, $\alpha = 0.0$) across 10 interpolation steps.

Ghostly Overlap vs. Smooth Structural Change

The key question for this experiment is whether the transition resembles a *ghostly overlap* (FCNN-style) or a *smooth structural change* (CNN-VAE style).

As seen in Figure 4, the result is clearly a **smooth structural change**, characteristic of a well-trained CNN-VAE. The observations are as follows:

- At $\alpha = 1.0$, the image closely resembles the original food image, with warm brown tones and recognisable food textures.
- As α decreases from 1.0 to 0.5, the warm tones gradually fade and the image becomes softer and more diffuse, with the food structure slowly dissolving.
- At $\alpha = 0.5$ (the midpoint), the image shows a blended intermediate — neither clearly food nor mountain, but a coherent mix of both colour palettes.
- From $\alpha = 0.4$ to $\alpha = 0.0$, cool blue and grey tones progressively emerge, eventually resolving into the snowy mountain landscape of Class B.

Discussion

This gradual colour and structural transition — from warm browns to cool blues — is the hallmark of a **CNN-VAE style** interpolation. The convolutional architecture preserves spatial coherence at every step, unlike an FCNN-VAE which would produce ghostly superimpositions of both classes simultaneously (e.g. food textures and mountain edges appearing on top of each other).

The smoothness is a direct result of the KL divergence regularisation, which forces the latent space to be continuous and well-structured, ensuring that points lying between two class representations decode into perceptually meaningful intermediate images. The intermediate frames, while blurry due to the compact $128 \times 1 \times 1$ latent bottleneck, remain coherent and visually interpretable throughout the transition.

2.2 Generating from Pure Noise

2.2.1 Problem Definition

A Variational Autoencoder (VAE) is a generative model that learns a structured, continuous latent space by training the encoder to map inputs to a Gaussian distribution rather than a fixed point. The KL-divergence regularization term forces this latent distribution to approximate a standard normal N(0,I). A key consequence of this property is that the decoder becomes a standalone generative model:once training is complete, the encoder is no longer needed. Novel samples can be produced by drawing random vectors directly from N(0, I) and passing them through the decoder alone.

To test this, we sampled 16 random latent vectors from

$$\mathbf{z} \sim \mathcal{N}(0, I)$$

and passed them through the trained decoder. The generated outputs were then arranged in a 4×4 grid. We performed this experiment for both MNIST and Tiny-ImageNet-10.

The generated MNIST images are clear and visually convincing. The decoder successfully maps random vectors from the prior into recognisable digit-like shapes across multiple classes. This suggests that the KL regularisation shaped the latent space well enough for the prior and posterior to overlap meaningfully.

2.2.2 Methodology

The experiment uses a convolutional VAE with a shared architecture across both datasets. MNIST images were upscaled to 64×64 and converted to 3-channel to match Tiny-ImageNet.

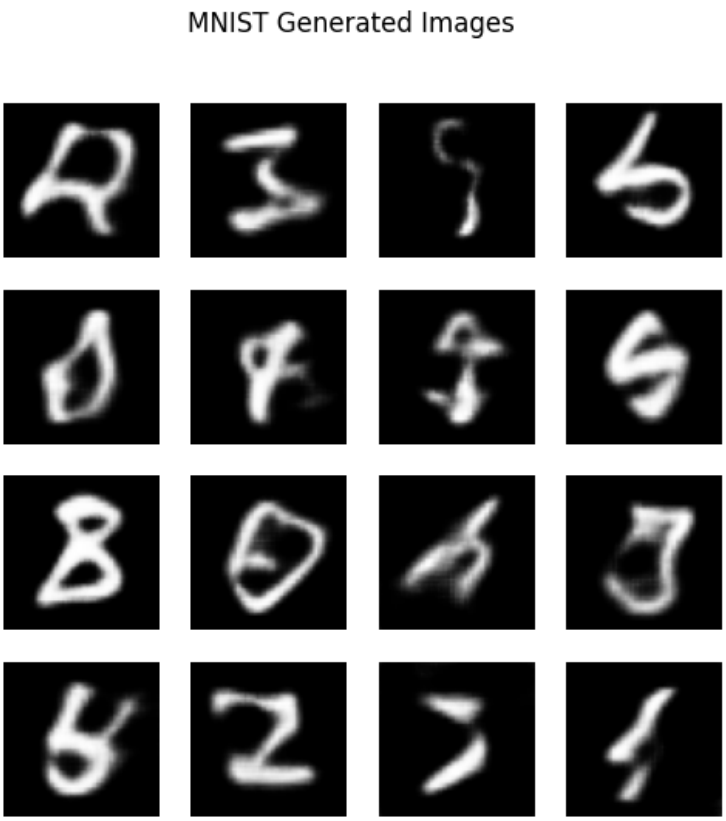
Hyperparameter	**MNIST**	**Tiny-ImageNet**	**Rationale**
Input size	64x64x3	64x64x3	Unified architecture
Latent dim	128	128	Sufficient capacity
Batch size	64	32	Memory constraint
Optimizer	Adam, 1e-3	Adam, 1e-3	Standard for VAEs
Epochs	5	5	Convergence demo
Beta	0.01	0.01	Prioritize sharpness
Recon loss	MSE	MSE	Stable gradients

Architecture:

- **Encoder:** 4× strided Conv2d (3→32→64→128→256, kernel=4, stride=2) → flatten → dual linear heads for μ and $\log \sigma^2$
- **Latent vector:** $z = \mu + \sigma \cdot \epsilon, \quad \epsilon \sim \mathcal{N}(0, I), \quad \dim = 128$
- **Decoder:** Linear projection → 4× ConvTranspose2d (256→128→64→32→3) → Sigmoid

2.2.3 Results and Visualizations

Generated Images From MNIST:



Generated Images From TinyImageNet:

Comparison and Discussion

The MNIST-trained decoder produces recognisable handwritten digits with clear strokes, good contrast, and digit-like curvature. This suggests that the model has successfully learned a structured latent manifold. The diversity of the 16 samples confirms that the latent space is spread across the full extent of N(0, I) rather than collapsed to a small region. This quality stems from three factors:

- MNIST is a low-complexity dataset with consistent style, scale, and background.
- 60,000 training images provide ample coverage of the digit manifold.
- The KL regularisation (even at low beta) successfully aligns the latent space with N(0, I).

The Tiny-ImageNet-trained decoder generates images that lack sharp object boundaries and identifiable content. While no clear objects are discernible, the colour distributions are plausible (sky blues, sandy tans, dark foliage tones) and the outputs are not random noise, resembling out-of-focus natural scene. This indicates the model has captured low-frequency statistics of natural images but has not learned a tight enough latent manifold to produce sharp objects. This reduced quality is attributable to:

- Only 3,500 training images.
- Tiny-ImageNet contains 200 object categories with large intra-class variance.

Overall, this experiment confirms the main generative property of VAEs: after training, the decoder can synthesise realistic-looking images directly from samples drawn from the standard

Normal prior, without any help from the encoder. For MNIST, this works very well and produces diverse, sharp digit-like outputs. For TinyImageNet, the same principle still holds, but the generated samples remain blurrier and less semantically clear because the dataset is more complex and the learning problem is much harder.

3 Architecting the Convolutional VAE (CVAE)

3.1 Handling the 3-Channel Complexity (high-dimensional RGB data)

Model Architecture

In this part, a Convolutional Variational Autoencoder (CVAE) was designed for RGB images from Tiny-ImageNet-10, where each input image has size $64 \times 64 \times 3$. Since the input is a three-channel color image, the model must capture both spatial structure and color relationships, which makes the reconstruction task more challenging than grayscale data.

The encoder was implemented using five convolutional layers:

$$3 \to 32 \to 64 \to 128 \to 256 \to 512$$

where each layer uses kernel size 4, stride 2, and padding 1. Because of stride-2 downsampling at every stage, the spatial dimensions are progressively reduced from 64×64 to 32×32, 16×16, 8×8, 4×4, and finally 2×2. This gives an encoder output of size $512 \times 2 \times 2$. After this, two separate convolution layers generate the mean (μ) and log-variance ($\log \sigma^2$), each of shape $128 \times 1 \times 1$, forming the latent distribution of the VAE.

The decoder mirrors this design using transposed convolution layers:

$$128 \to 512 \to 256 \to 128 \to 64 \to 32 \to 3$$

Again, kernel size 4, stride 2, and padding 1 are used so that the spatial resolution is restored from 1×1 back to 64×64. A `ReLU` activation is applied after each intermediate layer, while the final layer uses `Sigmoid` so that the reconstructed image remains in the valid range $[0, 1]$.

The observed shape flow through the implemented architecture is summarized in Table 2.

Table 2: Observed shape flow in the CVAE

Stage	**Output Shape**
Input $\mathbf{x}$	$1 \times 3 \times 64 \times 64$
Encoder feature map $\mathbf{h}_{enc}$	$1 \times 512 \times 2 \times 2$
Mean $\boldsymbol{\mu}$	$1 \times 128 \times 1 \times 1$
Log-variance $\log \boldsymbol{\sigma}^2$	$1 \times 128 \times 1 \times 1$
Latent vector $\mathbf{z}$	$1 \times 128 \times 1 \times 1$
Reconstruction $\hat{\mathbf{x}}$	$1 \times 3 \times 64 \times 64$

Overall, the model follows the transformation

$$64 \times 64 \times 3 \to 1 \times 1 \times 128 \to 64 \times 64 \times 3$$

which creates a strong bottleneck. This compact latent space forces the network to retain only the most important global structure and dominant color information, while finer textures and small details are harder to reconstruct. This behaviour is expected for high-dimensional RGB images compressed into a very small latent representation.

Model Training

As shown in Figure 5, the loss decreases steadily during training, indicating that the model gradually improves its reconstruction ability. The training and validation curves also remain reasonably close to each other throughout the process, which suggests *stable learning behaviour* without any major divergence. Toward the later epochs, the improvement becomes slower and the curves begin to flatten, which is expected because the compact latent bottleneck restricts how much image detail can be preserved and reconstructed.

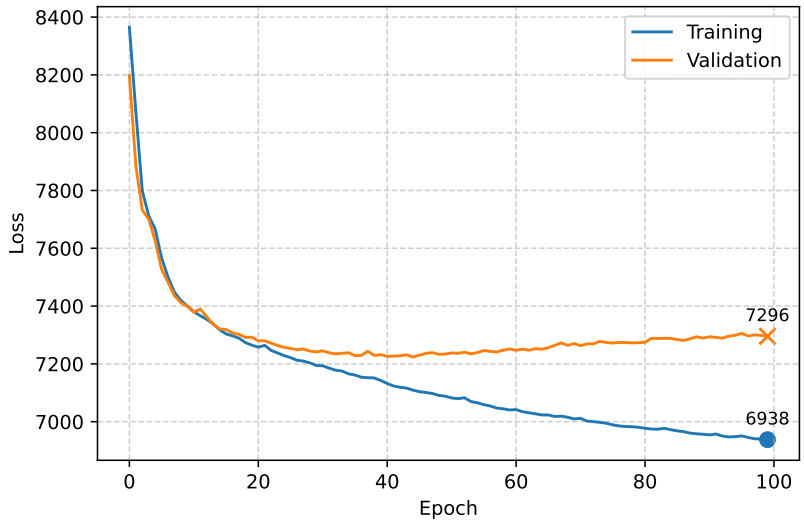


Figure 5: Training and validation loss

To check the model qualitatively, original and reconstructed test images were compared. These results are shown in Figure 6.

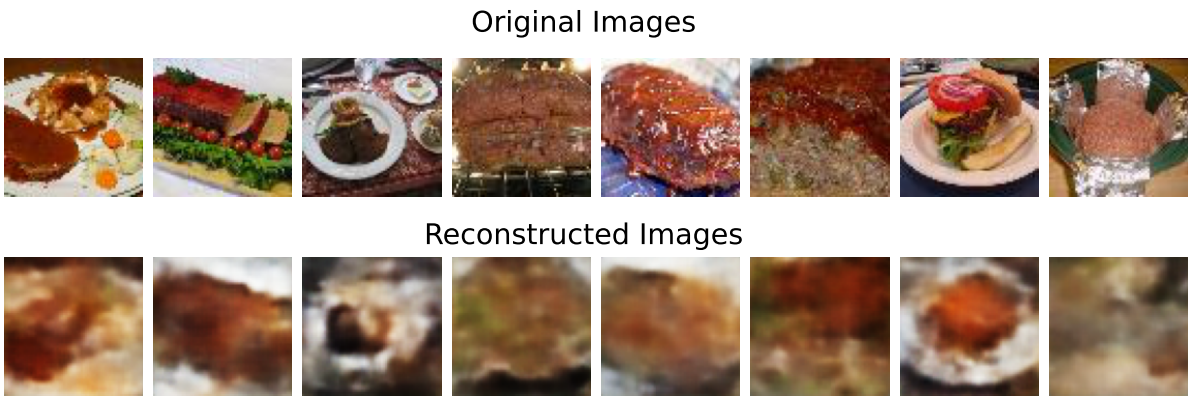


Figure 6: Original vs reconstructed test images

From the reconstruction results, a few things can be observed:

- The reconstructed images preserve the general object layout and dominant colors.
- The outputs are recognizable, which means the latent representation is meaningful.
- Fine textures and sharp boundaries are blurred, which is expected under such heavy compression.